CSCI 455 Robotics Studio

Project 2: Dialog Engine with Action Tags, Safety, and State Machine

Demo Day 1: Monday, March 9, 2026 for Group B
Demo Day 2: Friday, March 13, 2026 for Group A (Friday before Spring Break, you can demo early)

Overview

You will build a robust dialog engine that reads a TangoChat-style script file and runs a conversation. In addition to speaking text, the engine can trigger robot movement actions embedded as action tags in the script output. Your engine must be safe (interruptible, bounded-time actions, wheel deadman stop) and structured (explicit state machine).

Learning objectives

- Parse a small domain specific language (DSL) into a structured internal representation.
- Implement rule matching with scoped subrules and variable capture.
- Design an explicit finite state machine for interactive systems.
- Integrate with existing robot control primitives while enforcing safety constraints.
- Create deterministic behavior for grading and debugging.

Given

You are given:

- A sample TangoChat script file used for testing.
- Your Project 1 robot control code (Flask UI and movement primitives).
- A reference sheet describing the script language rules (See API Rules document).

Functional requirements

A. Script parsing (DSL) read in file on robot and parse

- Ignore blank lines.
- Ignore comments starting with # (everything from # to end of line).
- Accept inconsistent whitespace around tokens (spaces/tabs).
- Report parsing errors with: file name, line number, error category, and whether the error is fatal or non-fatal.
- On non-fatal errors: print/log the error and skip the line. Continue loading the rest of the file.

- On fatal errors (example: no valid top-level u: rules): stop and refuse to run.

B. Conversation rules and matching

- Keep using project 1 source code.
- Instead of using speech recognition for this one, you will add a text input box to your code from project 1. You will type in the user input, (simulating the human's speech) and send it to your flask server, which will then take the input string and handle all the rules below.

(The following rules are in the API document too with more in depth coverage, the action tags are not in the API)

- Rules are lines starting with u, u1, u2, ... followed by :(pattern):output.
- Matching is case-insensitive and should ignore basic punctuation in user input (at minimum: . , ! ?).
- Support bracket choices in patterns: [a b c "two words"] matches any option.
- Support bracket choices in outputs: [a b c "two words"] selects one option to speak (random selection required).
- Support definitions: ~name: [...] and allow ~name to appear inside patterns and outputs.
- Support variable capture: (my name is _) captures the wildcard text into the variable named on the output side, for example hello $name. If a variable is referenced but has never been set, speak "I don't know" for that variable.
- Scope rules: If u: matches and it has nested u1: rules, those u1 rules become active. If u1 matches and it has nested u2 rules, those become active, etc. If a top-level u: rule matches, it clears any previously active subruled starts a new scope.

C. Action tags

An output string may include zero or more action tags of the form <action_name>. Action tags are not spoken. They are executed in order after the text is produced.

All teams must implement all four required actions:

- <head_yes> - nod yes (tilt down then up then center, or closest equivalent).
- <head_no> - shake no (pan left then right then center, or closest equivalent).
- <arm_raise> - raise the arm to a clearly visible pose, hold briefly, then return to neutral, either arm, just a shoulder servo raise.
- <dance90> - excited dance step: rotate in place left about 90 degrees, then right about 90 degrees, then return to the starting heading (net zero).

Action tag rules:

- Action tags are enclosed in angle brackets with no spaces inside the tag.
- Multiple tags may appear in one output, for example: Great <head_yes> <arm_raise>.
- Unknown action tags must be logged as a warning and ignored (non-fatal).
- Action execution must not block the Flask request thread. Actions must run in a background worker or queue.

D. Safety requirements

- Global interrupt: if the user says stop, cancel, reset, or quit (case-insensitive), the engine must immediately stop wheels, stop any ongoing action sequence, clear scope, and return to IDLE.
- Bounded-time actions: each action must complete within a hard cap. Recommended caps: head actions <= 3 s, arm_raise <= 4 s, dance90 <= 6 s.
- Wheel deadman: any wheel movement must auto-stop even if an error occurs mid-action.
- Unmatched-while-in-scope guard: if you receive 4 consecutive inputs that match no active rule while in a nested scope, automatically reset to IDLE.
- Max nesting depth guard: if the script attempts to activate nesting deeper than 6, log an error and reset safely.

E. State machine + tracing (new)

You must implement and expose an explicit state machine. Minimum states:

- BOOT
- IDLE
- IN_SCOPE(k)
- EXEC_ACTIONS
- provide a command line flag --seed N that makes random output choices deterministic.

Deliverables

- You will demo all the gradeable aspects of your program on the date for your group.
- Turn in your reflection logs to Canvas

Integration guidance

Recommended architecture (not required, but strongly suggested):
1) DialogEngine: parsing + rule matching only. Produces (speak_text, actions).
2) ActionRunner: maps action names to robot primitives (head, arm, wheels). Executes via a queue.
3) Flask handlers: never block. They should enqueue work and return immediately.
This separation makes your engine unit-testable and keeps your server responsive.

Demo scripts (for grading)

I gave a demo script that you should use for testing and debugging. The point is to demonstrate: definitions, choices, variable capture, scopes, all 4 actions, and safety interrupts.

Rubric

Category	Points	What we look for
Parsing and validation	20	Correctly loads scripts; skips

		non-fatal bad lines with line-numbered errors; refuses to run on fatal conditions.
Rule matching and language features	35	Definitions (~name), bracket choices in input and output, case-insensitive matching, punctuation handling, variable capture and $var recall, correct scope activation/deactivation.
Action tags and robot integration	20	Correct extraction of <...> tags, all four required actions implemented and visibly correct, actions executed sequentially from outputs.
Safety and robustness	15	Global interrupt works immediately, bounded action durations, wheel deadman stop, unmatched-in-scope reset, max depth guard.
State machine	10	Explicit states, clean transitions,

Total: 100 points.

1. **Debug / Reflection Log** (ties to 20% in a different rubric in the syllabus, graded separately, everyone turns in one to CANVAS)
    - Dated entries for each time you worked on the project
    - What worked, where you ran into troubles, how you fixed troubles
    - Hardware issues vs software issues
    - Partner issues?

One incorrect assumption discovered

Submission

- Submit to Canvas: Reflection log.
- Be ready to demo live on both demo days. Your live demo must trigger all four required actions.
    - **A required "test DSL file"** (WE provide at demo time) that you must run at demo.
    - **A printed console log** (just normal prints) that shows key events to show code progress: rule matched, state change, action started/ended, stop interrupt

    - We will be running the program from the webpage to test during demo.
- The Journal logs of your work and preparation for this project, this will go toward your journal submission percentages .